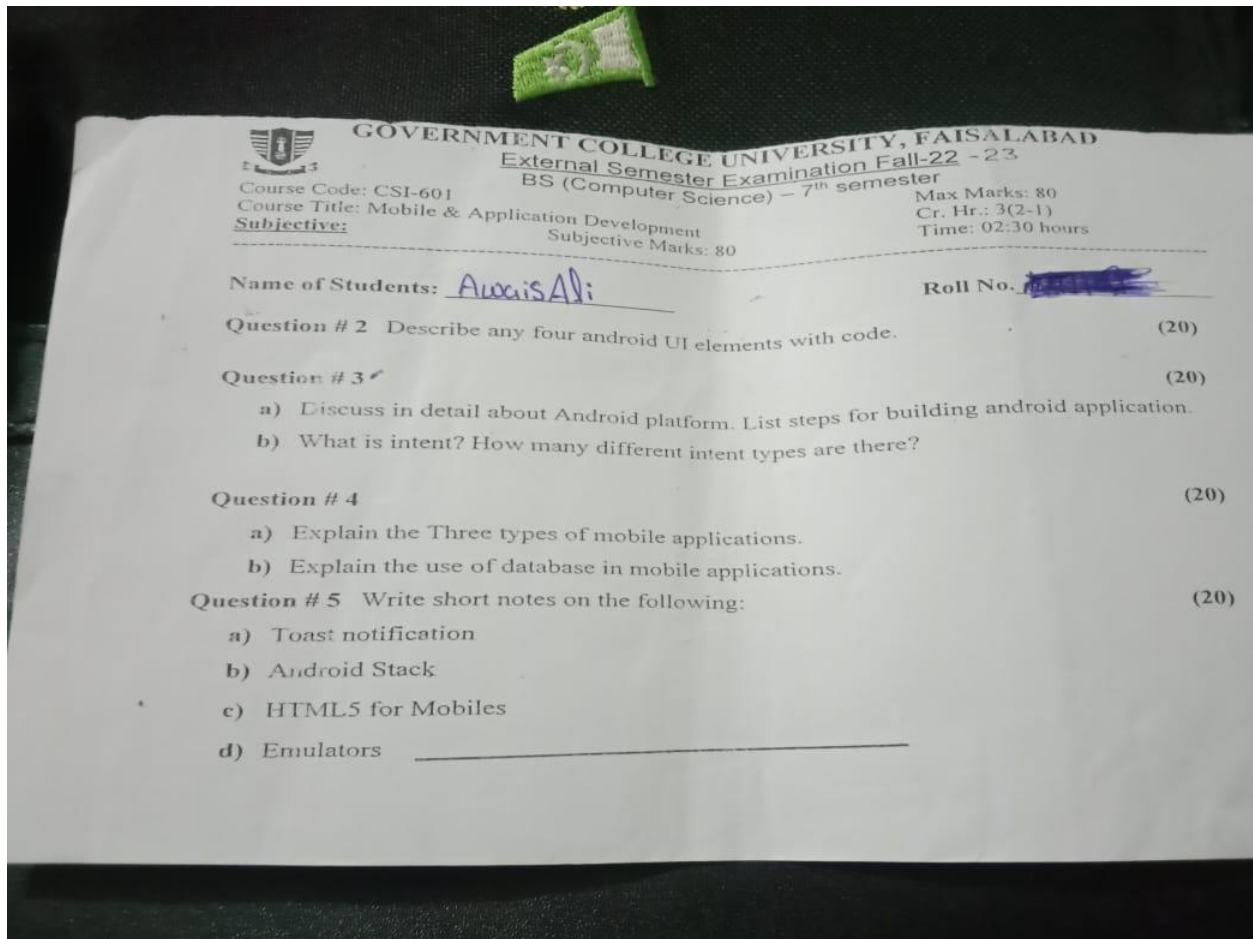




Q#1

Describe any four android UI elements with code.

In Android development, UI elements, also known as "widgets", are the building blocks for constructing user interfaces in an app. Here are four common Android UI elements along with example code snippets for each.

1. TextView

A TextView displays text to the user. You can modify attributes such as the font, color, and alignment.

<TextView

android:id="@+id/textViewExample"

android:layout_width="wrap_content"

android:layout_height="wrap_content"

```
android:text="Hello, World!"  
android:textSize="18sp"  
android:textColor="#FF0000"  
android:layout_marginTop="16dp"/>
```

2. Button

A Button is a UI element that users can click to perform actions.

```
<Button  
    android:id="@+id/buttonExample"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Click Me"  
    android:layout_marginTop="20dp"/>
```

3. EditText

An **EditText** allows the user to enter and modify text.

```
<EditText  
    android:id="@+id/editTextExample"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:hint="Enter your name"  
    android:inputType="text"/>
```

4. ImageView

An ImageView displays images. It can load images from various sources: drawable, bitmap, etc.

```
<ImageView  
    android:id="@+id/imageViewExample"  
    android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content"

android:src="@drawable/ic_launcher_background"

android:layout_marginTop="20dp"/>
```

5. CheckBox

A CheckBox allows the user to select or clear an option. It is typically used for displaying multiple options where the user can select more than one.

```
<CheckBox

    android:id="@+id/checkboxExample"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:text="Accept Terms and Conditions"

    android:layout_marginTop="10dp"/>
```

6. RadioButton

A RadioButton is used when you need the user to select one option from a set. It is usually combined within a RadioGroup.

```
<RadioGroup android:id="@+id/radioGroup"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content">

    <RadioButton

        android:id="@+id/radioButton1"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Option 1"/>

    <RadioButton

        android:id="@+id/radioButton2"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Option 2"/>

</RadioGroup>
```

7. Switch

A Switch is a two-state toggle switch widget that can select between two options.

<Switch

```
    android:id="@+id/switchExample"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Enable Notifications"
    android:layout_marginTop="10dp"/>
```

8. ProgressBar

A ProgressBar is a visual indicator of progress in some operation. It can be either determinate or indeterminate.

<ProgressBar

```
    android:id="@+id/progressBarExample"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"/>
```

Q#2

a) Discuss in detail about Android platform. List steps for building android application.

Android is a comprehensive operating system developed by Google, based on the Linux kernel, and designed primarily for touchscreen mobile devices such as smartphones and tablets. It's the most widely used mobile OS in the world and has expanded to other platforms like wearables (Android Wear), TVs (Android TV), and even automobiles (Android Auto).

Key Features of Android:

1. **Open Source:** Android is open-source, which means it offers an accessible environment for developers to create applications, contributing to its vast ecosystem.
2. **Massive User Base:** With its widespread adoption, apps developed on Android can reach a broad audience.

3. **Google Play Store:** The primary app store, making it easy for developers to distribute and monetize their applications.
4. **Multitasking:** Android supports multitasking and seamless app switching.
5. **Customization:** Highly customizable for both end users and manufacturers.
6. **Integration with Google Services:** Seamless integration with a wide array of Google services and APIs like Google Maps, Google Drive, and cloud-based services which enhance app functionality.

List of Android Versions and Initial Stable Release Dates



Android 1.0
September 23, 2008



1.5 - Cupcake
April 27, 2009



1.6 - Donut
September 15, 2009



2.0/2.1 - Éclair
October 26, 2009



2.2 - Froyo
May 20, 2010



2.3 - Gingerbread
December 6, 2010



3.0 - Honeycomb
February 22, 2011



4.0 - Ice Cream Sandwich
October 18, 2011



4.1/4.3 - Jelly Bean
July 9, 2012



4.4 - KitKat
October 31, 2013



5.0 - Lollipop
November 12, 2014



6.0 - Marshmallow
October 5, 2015



7.0 - Nougat
August 22, 2016



8.0 - Oreo
August 21, 2017



9.0 - Pie
August 6, 2018



Android 10
September 3, 2019



Android 11
September 8, 2020



Android 12
October 17, 2021

Steps for Building an Android Application

Step 1: Set Up Android Development Environment

- **Install Android Studio:** Download and install Android Studio, which includes the IDE, the Android SDK tools, and the Android platform with a virtual device to test your app.
- **Configure Android SDK:** Inside Android Studio, configure the specific Android SDK versions and tools.

Step 2: Create a New Project

- **Start a New Project:** Choose 'Start a new Android Studio project'—select your project's name, namespace, and initial configuration.
- **Select Device Compatibility:** Choose which types of devices (e.g., phone, tablet) your app will support.

Step 3: Design the User Interface

- **Use Layout Editor:** Design your app's layout using Android Studio's Layout Editor. You can drag and drop components like buttons, text views, and images.
- **Edit XML:** For more detailed customization, edit the XML files manually to define your UI elements.

Step 4: Write the Application Code

- **Activity Coding:** Write the code for your app's activities. Activities represent single screens in an app.
- **Implement User Interaction:** Code the logic for user interactions within your activities using Java or Kotlin.

Step 5: Add App Resources

- **Include Graphics:** Add graphic elements like icons and splash screens.
- **Localization:** Add different languages and resources for localization if necessary.

Step 6: Test the Application

- **Use the Emulator:** Run your application on the Android Emulator that comes with Android Studio to simulate different devices.
- **Physical Device Testing:** Deploy your app on physical Android devices to ensure it operates smoothly.

Step 7: Optimize Your Application

- **Performance Tuning:** Utilize Android's profiling tools to optimize performance and battery life.
- **Security Enhancements:** Ensure your application follows security best practices to protect user data.

Step 8: Prepare for Release

- **Versioning:** Set your application's version in your app's manifest file.
- **Build APK or AAB:** Generate the APK (Android Package) or AAB (Android App Bundle) file to distribute your application.

Step 9: Publish to the Google Play Store

- **Google Play Console:** Set up a developer account on the Google Play Console, submit your APK or AAB, and fill out the listing details for your app.
- **Distribution:** After approval, your app will be available on the Google Play Store for users to download and install.

Step 10: Maintain and Update

- **Monitor Feedback:** Pay attention to user feedback and app performance to identify areas for improvement.
- **Update Regularly:** Provide updates to add features, fix bugs, and improve user experience.

b) What is intent? How many different intent types are there?

Android Intent is the message that is passed between components such as activities, content providers, broadcast receivers, services etc. It is generally used with `startActivity()` method to invoke activity, broadcast receivers etc. The dictionary meaning of intent is intention or purpose. So, it can be described as the intention to do action.

Android intents are mainly used to:

- Start the service
- Launch an activity
- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.

Types of Intents

There are two primary types of Intents in Android:

1. Explicit Intents

Explicit intents are used when you want to start a specific component (such as an activity or service) directly within your application or another application where you know the exact class name of the component to be launched. This type of intent is very direct; you specify which component to start by providing its fully qualified class name.

Example:

If you have a button in an app that, when clicked, should open a new screen for sending messages, you would use an explicit intent. You would create the intent by specifying the class name of the activity that handles sending messages.

Code Example:

```
Intent intent = new Intent(context, SendMessageActivity.class);  
  
startActivity(intent);
```

2. Implicit Intents

Implicit intents, unlike explicit ones, do not specify the component. Instead, they declare a general action to perform, which allows any app component that filters for that action (and satisfies other criteria like data type) to handle the intent. This is useful for integrating with components of other applications without direct interaction with their internal structure.

Example:

Suppose you want to show the user a location on a map. Instead of creating an explicit intent for a specific map activity, you create an implicit intent that simply describes the action to view a map.

Code Example:

```
Intent intent = new Intent(Intent.ACTION_VIEW);  
  
intent.setData(Uri.parse("geo:0,0?q=1600+Amphitheatre+Parkway,+Mountain+View,+California"));  
  
startActivity(intent);
```

Q#3

a) Explain the Three types of mobile applications.

Native apps

These are designed for specific mobile operating systems like Android, IOS, or Windows. Here, Android is developed using Java and IOS uses Swift or Objective C. They are published on Play Store or App Store.

Examples: Google Maps, Spotify, Telegram, WhatsApp, etc...

Technologies used: C++, Java, Kotlin, Objective-C, Python, React Native, Swift.

Advantages:

1. They have best performance.
2. More secure.
3. These are user interactive.
4. They have access to full features of device.

Disadvantages:

1. They are developed only on single platform.

2. These are expensive to maintain.

Web Apps

Mobile web applications are optimized for mobile devices and they are accessible through a web browser on a smartphone. These are developed using HTML/CSS/Javascript. They runs with the help of web browser applications like chrome, safari, Firefox etc.

Examples: Amazon, Canva, Netflix, Walmart etc.

Technologies used: C#, C++, HTML, CSS, Javascript, Java, Perl, PHP, Ruby, Python, Typescript.

Advantages:

1. It is easy to development.
2. Easy to maintain.
3. These are cheaper than Native and Hybrid apps.
4. They works on all platforms.

Disadvantages:

1. These are device browser dependent.
2. Poor user experience.
3. Slow performance.
4. These are less interactive with device utilities.

Hybrid apps

Hybrid apps combine elements of both native and web apps, using a single codebase to run on multiple platforms. These are deployed on container that uses mobile WebView object.

Examples: Facebook, Gmail, Instagram, LinkedIn, Twitter, Uber, Yelp.

Technologies Used: HTML5, CSS, Flutter, JavaScript, Appy Pie, AppMakr, Ionic, Objective-C, Swift, React Native, Wrap-Up.

Advantages:

1. These are cheaper to maintain.
2. These are easy to develop.
3. They have access to features of device.
4. Faster development due to single codebase.

Disadvantages:

1. These are slower as compared to native apps.
2. Less interactive than native apps.

c) Explain the use of database in mobile applications

Databases in mobile applications play a crucial role in managing data efficiently and effectively. Here's a breakdown of how they are used and why they are important:

1. Data Storage

Mobile applications often need to store data locally to allow users to operate the app without an internet connection. Databases provide a structured way to store data directly on the device. This can include user preferences, app settings, and data collected during offline use.

2. Data Synchronization

When the device connects to the internet, the database can synchronize its local data with a remote server. This ensures that data is consistent across multiple devices or sessions. For example, a notes app might sync notes across a user's phone and tablet.

3. User Experience

Databases enhance the user experience by enabling quick searches, filtering data, and retrieving records efficiently. This responsiveness is crucial for maintaining smooth interaction within the app, especially for features like searching contacts, managing emails, or browsing product catalogs.

4. Offline Capabilities

By storing data locally, mobile apps can operate in offline mode, making them more reliable and versatile. Users can continue to access features and data without an internet connection, which is particularly useful in areas with poor connectivity or for reducing data usage.

5. Security

Databases can also help in implementing security measures at the data storage level. Data encryption and secure access mechanisms can protect sensitive information stored on the device from unauthorized access.

6. Scalability

As the amount of data grows, a well-designed database schema helps in maintaining performance and scalability. It allows the app to handle an increased load with minimal changes to the database structure.

Q#4

Write short notes on the following

1. Toast notification

A Toast notification is a small message that briefly appears on the screen of an Android device. It's used in apps to display quick alerts that inform the user but don't require them to take any action—

meaning the app doesn't pause or wait for the user to tap on the notification. Instead, it shows up over whatever they're doing, and after a few seconds, it disappears on its own.

How Does a Toast Work?

Duration:

A Toast can last either a short time (about 2 seconds) or a longer time (about 3.5 seconds). You decide how long it should stay visible when you create it.

Position:

While Toasts typically pop up at the bottom of the screen, you can change this. If you prefer, you can make them appear at the top or in the middle of the screen.

No Interaction:

The user doesn't need to interact with a Toast. There's no need to dismiss it; it goes away by itself. This makes Toasts unobtrusive; they tell the user something useful but don't get in the way of their experience.

Example of Using a Toast:

```
Toast.makeText(context, "Your file has been saved", Toast.LENGTH_SHORT).show();
```

2. Android Stack

The Android Stack, also known as the Android software stack, refers to the comprehensive set of software components that power devices running the Android OS. It is structured in layers, starting from the lowest level of the operating system up to the applications at the top. Here's a brief look at each layer within the stack:

Linux kernel :

This is the kernel on which Android is based. This layer contains all the low level device drivers for the various hardware components of an Android device.

Platform Libraries

These contain all the code that provides the main features of an Android OS. For example, the SQLite library provides database support so that an application can use it for data storage. The WebKit library provides functionalities for web browsing.

Android runtime

At the same layer as the libraries, the Android runtime provides a set of core libraries that enable developers to write Android apps using the Java programming language. The Android runtime also includes the Dalvik virtual machine, which enables every Android application to run in its own process. Dalvik is a specialized virtual machine designed specifically for Android and optimized for battery-powered mobile devices with limited memory and CPU.

Application framework

Exposes the various capabilities of the Android OS to application developers so that they can make use of them in their applications.

Applications

At this top layer, you will find applications that send with the Android device (such as Phone, Contacts, Browser, etc.), as well as applications that you download and install from the Android Market. Any applications that you write are located at this layer.

3. HTML5 for Mobiles

HTML5 has significantly impacted the landscape of mobile application development by providing a versatile and platform-independent solution for creating mobile apps. It serves as a core technology in the development of web-based mobile applications. Here's an overview of its role and benefits in mobile app development:

1. Cross-Platform Compatibility:

HTML5 makes it easier to develop applications that run seamlessly across multiple mobile platforms (like iOS, Android, and Windows Phone) without needing to be rewritten or adapted for each specific operating system. This is because HTML5 apps run within the mobile browser, leveraging the standard web technologies - HTML, CSS, and JavaScript.

2. Access to Device Features:

Although HTML5 primarily runs in a web browser, advancements in mobile browsers and associated APIs (like Geolocation API, Canvas API, and Web Storage API) have increased the accessibility to device-specific features. These APIs allow HTML5 mobile apps to access device capabilities such as GPS, accelerometer, and local storage, enhancing the user experience to be closer to that of native apps.

3. Offline Capabilities:

HTML5 supports local storage and offline capabilities through the Web Storage API and the Application Cache. This allows users to access certain features of the app without an internet connection, by storing necessary data and resources locally on the device.

4. Improved Semantics and Accessibility:

HTML5 introduces more descriptive HTML tags (like `<header>`, `<footer>`, `<section>`, and `<article>`) which not only improve the semantic structure of the pages but also aid in accessibility. This is particularly beneficial in mobile contexts where accessibility features can significantly enhance user interactions on smaller devices.

5. Enhanced Multimedia Support:

HTML5 provides built-in support for embedding rich multimedia content such as audio and video directly into HTML code using `<audio>` and `<video>` tags, without requiring external plugins or third-party software. This streamlined approach is ideal for mobile devices, where simplicity and speed are crucial.

6. Performance and Optimization:

While HTML5 applications are generally slower than native apps, there have been continuous improvements in mobile browser performance, which narrow this gap. Techniques like efficient JavaScript execution and hardware-accelerated CSS transformations help optimize the performance of HTML5 applications on mobile devices.

4.Emulators

Emulators are software tools that allow developers to simulate a mobile device on a computer. They are an essential part of the mobile application development process because they provide an environment to test and debug applications without needing physical devices. Here are some key points about emulators:

1. Device Simulation:

Emulators mimic the hardware and software environments of mobile devices, such as smartphones and tablets. They simulate various device functions including hardware characteristics (like screen size and resolution), operating systems, and API behaviors.

2. Development and Testing:

Emulators are particularly useful in the early stages of development when developers need to quickly test and iterate their applications. They allow developers to test how an app would run on different devices and operating systems without the need for actual hardware.

3. Cost-Effective:

Using emulators can be a cost-effective approach because it reduces the need for multiple physical devices during testing. Developers can emulate different devices and configurations on a single computer, saving on hardware costs.

4. Limitations:

While emulators provide a valuable testing environment, they do not perfectly replicate the hardware performance of actual devices. Issues such as differences in processing power, battery usage, and other hardware-specific characteristics might not be accurately reflected. Therefore, while emulators are good for initial testing, it's important to also test applications on physical devices to ensure they perform well in real-world conditions.

5. Popular Emulators:

There are several popular emulators used in mobile development. For Android development, Google provides an official Android Emulator that is part of the Android Studio IDE. Apple offers the Simulator app as part of Xcode for iOS development. These tools support a variety of device types and configurations.



GOVERNMENT COLLEGE UNIVERSITY, FAISALABAD

MID TERM EXAMINATIONS (FALL SEMESTER 2022)

OBJECTIVE

Program: Bachelor of Computer Science

Semester: 7th

Roll No. 1139

Course Code: CSI-601

Course Title: Mobile & App. Development

Time Allowed: 50 min

Marks: 8

Session: 2019-23

Instruction: Attempt all Questions

Q. No. 2 Answer the following short questions. (4)

- (1) What is android? Discuss features of android.
- (2) What is format of id to any widget in XML? Write syntax in xml.

Q. No. 3 Develop an android application user interface in XML as shown in Fig. In java file develop logic to change text of a Text View as given in Edit Text. (4)



Q#1

What is android? Discuss features of android.

Android is an open source and Linux-based Operating System for mobile devices such as smartphones and tablet computers. Android was developed by the Open Handset Alliance, led by Google, and other companies. Android offers a unified approach to application development for mobile devices which means developers need only develop for Android, and their applications should be able to run on different devices powered by Android.

The first beta version of the Android Software Development Kit (SDK) was released by Google in 2007 where as the first commercial version, Android 1.0, was released in September 2008.

Features of Android

Beautiful UI

Android OS basic screen provides a beautiful and intuitive user interface.

Connectivity

GSM/EDGE, IDEN, CDMA, EV-DO, UMTS, Bluetooth, Wi-Fi, LTE, NFC and WiMAX.

Storage

SQLite, a lightweight relational database, is used for data storage purposes.

Media support

H.263, H.264, MPEG-4 SP, AMR, AMR-WB, AAC, HE-AAC, AAC 5.1, MP3, MIDI, Ogg Vorbis, WAV, JPEG, PNG, GIF, and BMP.

Messaging

SMS and MMS

Web browser

Based on the open-source WebKit layout engine, coupled with Chrome's V8 JavaScript engine supporting HTML5 and CSS3.

Multi-touch

Android has native support for multi-touch which was initially made available in handsets such as the HTC Hero.

Multi-tasking

User can jump from one task to another and same time various application can run simultaneously.

Resizable widgets

Widgets are resizable, so users can expand them to show more content or shrink them to save space.

Multi-Language

Supports single direction and bi-directional text.

Wi-Fi Direct

A technology that lets apps discover and pair directly, over a high-bandwidth peer-to-peer connection.

Android Beam

A popular NFC-based technology that lets users instantly share, just by touching two NFC-enabled phones together.

What is format of id to any widget in XML? Write syntax in xml.

In Android development, when you define widgets (like buttons, text views, etc.) in an XML layout file, you typically assign each widget a unique identifier. This identifier is used to reference the widget from your Java or Kotlin code. The identifier is set using the `android:id` attribute within the widget's XML tag.

syntax

```
<WidgetType  
    android:id="@+id/your_unique_identifier_name"  
    ... additional attributes ...>  
</WidgetType>
```

Example

```
<TextView  
    android:id="@+id/textViewExample"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Hello World!" />
```

In this example:

- `WidgetType` is replaced by `TextView`, which is the type of the widget.
- `android:id` is the attribute used to specify the identifier.
- `@+id/textViewExample` is the value for the ID, where:
 - `@+id/` tells Android to create a new ID resource.
 - `textViewExample` is the name of the ID, which should be unique within the XML file.

Develop an android application user interface in XML as shown in Fig. In java file develop logic to change text of a Text View as given in Edit Text.

XML Layout (activity_main.xml):

```
<?xml version="1.0" encoding="utf-8"?>  
  
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"
```



```
android:layout_height="match_parent">
```

```
<TextView
```

```
    android:id="@+id/textView"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Fragment: Example"
```

```
    android:layout_centerHorizontal="true"
```

```
    android:layout_marginTop="16dp"/>
```

```
<EditText
```

```
    android:id="@+id/editText"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:hint="Change Text"
```

```
    android:layout_below="@id/textView"
```

```
    android:layout_marginTop="16dp"/>
```

```
<Button
```

```
    android:id="@+id/buttonChangeText"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Change Text"
```

```
    android:layout_below="@id/editText"
```

```
    android:layout_centerHorizontal="true"
```

```
    android:layout_marginTop="16dp"/>
```

```
</RelativeLayout>
```

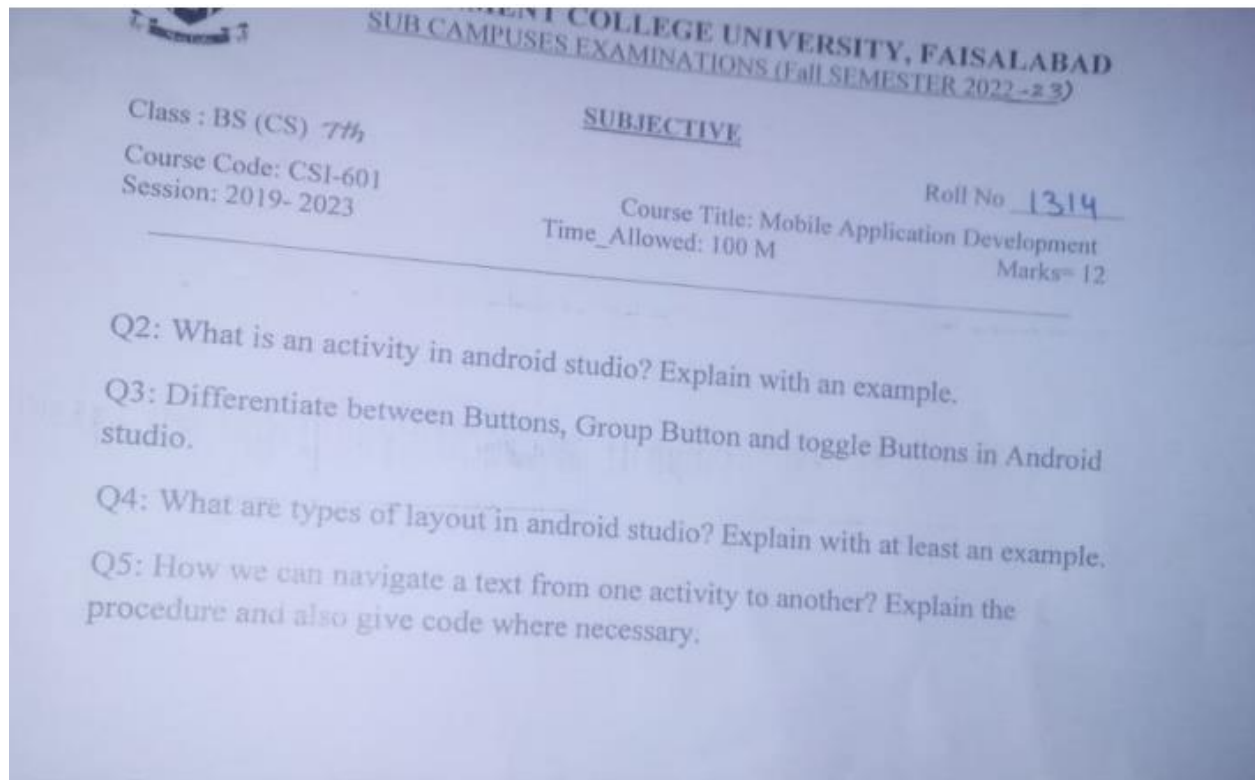
Java Activity (MainActivity.java):

```
import android.os.Bundle;
```

```
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    private TextView textView;
    private EditText editText;
    private Button buttonChangeText;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        textView = findViewById(R.id.textView);
        editText = findViewById(R.id.editText);
        buttonChangeText = findViewById(R.id.buttonChangeText);

        buttonChangeText.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                // Set the TextView text to the EditText content
                textView.setText(editText.getText().toString());
            }
        });
    }
}
```



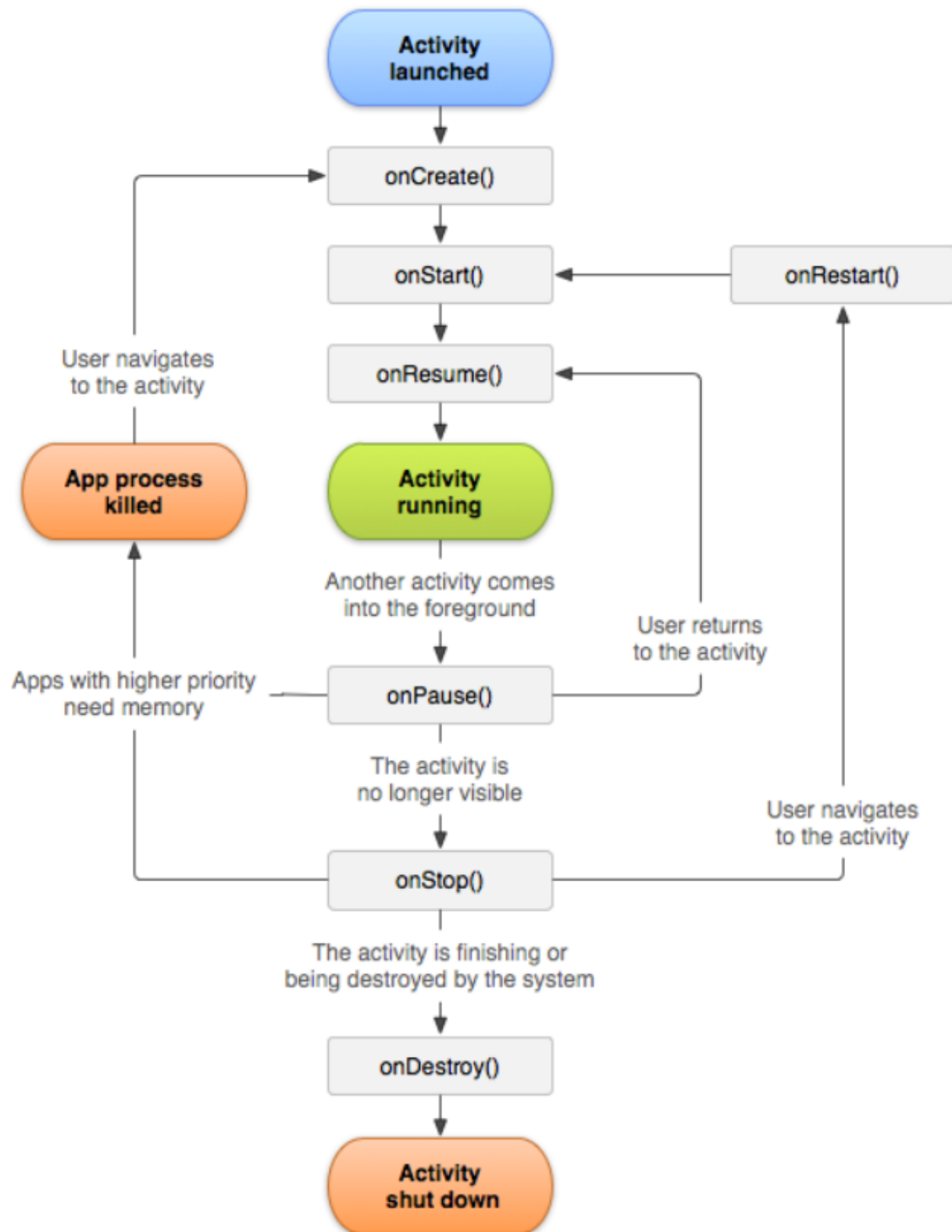
What is an activity in android studio? Explain with an example.

An activity is one screen of an app. In that way the activity is very similar to a window in the Windows operating system. The most specific block of the user interface is the activity. An Android app contains activities, meaning one or more screens.

Examples: Login screen, sign up screen, and home screen.

LIFECYCLE OF ACTIVITY

An activity goes through a number of states. Activity lifecycle in Android manages the different states of activity, such as when the activity starts or stops, etc. All of these states are managed by callback methods.



There are seven callback methods:-

onCreate():- This callback method must be created by you, this is fired when your activity is created by the system. For example, when you open the app it will call onCreate(), onStart(), and onResume() methods. These three methods are initiated when you open an app.

onStart():- There should always be an onStart() callback after onCreate() finishes. In the started state users will be able to see the activity but they are not ready for user interaction.

onResume():- In this, activities will be in the foreground and ready for user interaction. You can understand the use of this callback from this example: "when the user presses the phone's answer button it will give onPause() after you end up calling it will again give onResume()".

onPause():- This callback occurs when there is a case of another activity starting in front of the current activity. It is the opposite of onResume(). An example for this callback can be "when you open the app, that is another app from the notification bar, or open settings then it will call onPause() and onStop()".

onStop():- It is the opposite of onStart(). In this case, activity will no longer be visible to the users.

onRestart():- This occurs when the activity is stopped and before the activity starts again. It is a less common callback.

onDestroy():- It is opposite of onCreate(). This starts when the system needs to free memory or when finish() is called. It is used for cleanup which is a very common activity.

```
import android.os.Bundle;
import android.app.Activity;
import android.util.Log;

public class MainActivity extends Activity {
    String msg = "Android : ";

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(msg, "The onCreate() event");
    }

    @Override
    protected void onStart() {
        super.onStart();
        Log.d(msg, "The onStart() event");
    }
}
```

```

    }

    @Override
    protected void onResume() {
        super.onResume();
        Log.d(msg, "The onResume() event");
    }

    @Override
    protected void onPause() {
        super.onPause();
        Log.d(msg, "The onPause() event");
    }

    @Override
    protected void onStop() {
        super.onStop();
        Log.d(msg, "The onStop() event");
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        Log.d(msg, "The onDestroy() event");
    }
}

```

Differentiate between Buttons, Group Button and toggle Buttons in Android studio.

1. Button

A Button in Android is the simplest form of a button UI element used to perform an action when clicked. It typically triggers some processing or navigation in the app. Buttons are customizable and can include text or icons.

- **Usage:** Used for initiating an action like submitting a form, opening a new screen, etc.

XML Example:

```
<Button
```

```
    android:id="@+id/button"
```

```
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Click Me" />
```

- **Customization:** Can be styled with different colors, shapes, and effects.

2. Button Group (MaterialButtonToggleGroup)

A Button Group, specifically in the Material Design components, is known as **MaterialButtonToggleGroup**. This group is used to create a segmented control group, where multiple buttons are grouped together. Only one button in the group can be active at a time, making it useful for selecting one option from multiple choices.

- **Usage:** Suitable for exclusive selection scenarios, like choosing a type of payment method.

XML Example:

```
<com.google.android.material.button.MaterialButtonToggleGroup
    android:id="@+id/toggleButtonGroup"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:singleSelection="true">

    <Button
        android:id="@+id/option1"
        android:text="Option 1"
        style="?attr/materialButtonOutlinedStyle"/>
    <Button
        android:id="@+id/option2"
        android:text="Option 2"
        style="?attr/materialButtonOutlinedStyle"/>
</com.google.android.material.button.MaterialButtonToggleGroup>
```

- **Customization:** Buttons within the group can have consistent styling or differentiated styles.

3. Toggle Button

A Toggle Button in Android is a special type of button that holds a state: checked or unchecked. It's typically used for toggling a feature on or off, such as Wi-Fi or Bluetooth settings. This button visually indicates its current state.

- **Usage:** Best for switching settings or features on or off.

XML Example:

```
<ToggleButton
    android:id="@+id/toggleButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textOn="ON"
    android:textOff="OFF" />
```

- **Customization:** Can display different texts or colors depending on the state

What are types of layout in android studio? Explain with at least an example.

In Android Studio, layouts are used to define the structure and appearance of the user interface (UI) of an Android application. There are several types of layouts available, each serving different purposes. Here are some common types:

1. Linear Layout:

In a linear layout, things are laid out either horizontally or vertically, like lining up blocks in a row or stacking them on top of each other.

Example :

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
```



```
</LinearLayout>
```

2. Relative Layout:

In a relative layout, things are placed relative to each other, like saying something should be to the right of another thing.

Example:

```
<RelativeLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">  
</RelativeLayout>
```

3. Constraint Layout

In a constraint layout, you can position things by setting rules between them, like saying something should be below or to the right of another thing.

Example :

```
<androidx.constraintlayout.widget.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">  
</androidx.constraintlayout.widget.ConstraintLayout>
```

4. Frame Layout:

In a frame layout, things are placed one on top of the other, like putting pictures on top of each other.

Example:

```
<FrameLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent">  
</FrameLayout>
```

5. Table Layout:

In a table layout, things are arranged in rows and columns, like making a table in a word document.

Example :

```
<TableLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">  
</TableLayout>
```

6. Grid Layout:

- Definition: In a grid layout, things are arranged in a grid-like structure with rows and columns, like arranging items on a checkerboard.

Example:

```
<GridLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">  
</GridLayout>
```

7. Scroll View:

- Definition: A scroll view allows you to put more things on the screen than can fit at once, and the user can scroll up and down or left and right to see the rest.

Example :

```
<ScrollView  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"
```

```
        android:layout_height="match_parent">
</ScrollView>
```

8. List View / RecyclerView:

- Definition: These are used for displaying a list of items that can be scrolled through. RecyclerView is more efficient for handling large lists.

Example:

```
<androidx.recyclerview.widget.RecyclerView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

9. Drawer Layout:

- Definition: A drawer layout is used for creating a side navigation panel that can be pulled in and out of view.

- Example:

```
<androidx.drawerlayout.widget.DrawerLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:layout_gravity="start">
    </LinearLayout>
</androidx.drawerlayout.widget.DrawerLayout>
```

10. Coordinator Layout:

- Definition: A coordinator layout is like a smart frame layout that helps coordinate the animations and interactions between different UI components.

- Example Code:

```
<androidx.coordinatorlayout.widget.CoordinatorLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent">  
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

How we can navigate a text from one activity to another? Explain the procedure and also give code where necessary.

Navigating with text from one activity to another in Android involves passing data using Intents. An Intent is a messaging object you can use to request an action from another app component. For passing data, you generally use Intent.putExtra() method to store the data in the Intent and then retrieve it in the second activity using getIntent().getExtras().

Here's a step-by-step procedure followed by an example in code:

Step-by-Step Procedure

1. **Define Two Activities:** You need at least two activities. Let's call them **FirstActivity** and **SecondActivity**.
2. **Modify AndroidManifest.xml:** Ensure both activities are declared in your **AndroidManifest.xml**.
3. **Pass Data using Intent:** In your **FirstActivity**, create an Intent to start **SecondActivity**. Add the text data to this Intent using **putExtra()**.
4. **Receive Data in Second Activity:** In **SecondActivity**, get the Intent that started it and retrieve the data.
5. **Update UI:** Use the retrieved data to update the UI in **SecondActivity**, such as displaying the text in a TextView.

FirstActivity.java	SecondActivity.java
import android.app.Activity; import android.content.Intent; import android.os.Bundle; import android.view.View; import android.widget.Button;	import android.app.Activity; import android.content.Intent; import android.os.Bundle; import android.widget.TextView;

<pre> import android.widget.EditText; public class FirstActivity extends Activity { private EditText editText; private Button button; @Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); setContentView(R.layout.activity_first); editText = findViewById(R.id.edit_text); button = findViewById(R.id.send_button); button.setOnClickListener(new View.OnClickListener() { @Override public void onClick(View v) { String textToSend = editText.getText().toString(); Intent intent = new Intent(FirstActivity.this, SecondActivity.class); intent.putExtra("text_key", textToSend); startActivity(intent); } }); } </pre>	<pre> public class SecondActivity extends Activity { private TextView textView; @Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); setContentView(R.layout.activity_second); textView = findViewById(R.id.text_view); // Get the Intent that started this activity // and extract the string Intent intent = getIntent(); String receivedText = intent.getStringExtra("text_key"); // Use the string in your UI textView.setText(receivedText); } } </pre>
FirstActivity Layout (activity_first.xml)	SecondActivity Layout (activity_second.xml)
<pre> <?xml version="1.0" encoding="utf-8"?> <LinearLayout xmlns:android="http://schemas.android.com/ apk/res/android" android:layout_width="match_parent" android:layout_height="match_parent" android:orientation="vertical" android:padding="16dp"> <EditText android:id="@+id/edit_text" android:layout_width="match_parent" </pre>	<pre> <?xml version="1.0" encoding="utf-8"?> <LinearLayout xmlns:android="http://schemas.android.com/ apk/res/android" android:layout_width="match_parent" android:layout_height="match_parent" android:orientation="vertical" android:padding="16dp" android:gravity="center"> <TextView android:id="@+id/text_view" </pre>

<pre> android:layout_height="wrap_content" android:hint="Enter text here"/> <Button android:id="@+id/send_button" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="Send to Second Activity" android:layout_gravity="center_horizontal" android:marginTop="20dp"/> </LinearLayout> </pre>	<pre> android:layout_width="wrap_content" android:layout_height="wrap_content" android:textSize="18sp" android:text="Default Text"/> </LinearLayout> </pre>
---	---

AndroidManifest.xml

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android"

    package="your.package.name">

    <application

        android:allowBackup="true"

        android:icon="@mipmap/ic_launcher"

        android:label="@string/app_name"

        android:roundIcon="@mipmap/ic_launcher_round"

        android:supportsRtl="true"

        android:theme="@style/AppTheme">

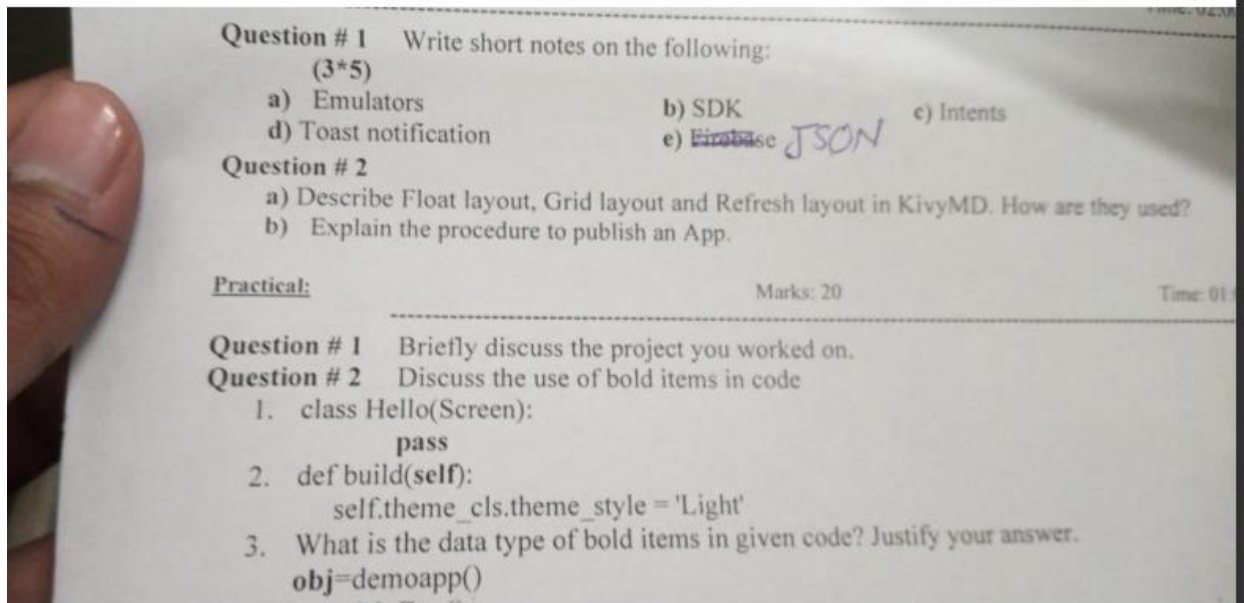
        <activity android:name=".FirstActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".SecondActivity" />

```

</application>
</manifest>



Question #1 Write short notes on the following

a.Emulators

Emulators are software tools that allow developers to simulate a mobile device on a computer. They are an essential part of the mobile application development process because they provide an environment to test and debug applications without needing physical devices. Here are some key points about emulators:

1. Device Simulation:

Emulators mimic the hardware and software environments of mobile devices, such as smartphones and tablets. They simulate various device functions including hardware characteristics (like screen size and resolution), operating systems, and API behaviors.

2. Development and Testing:

Emulators are particularly useful in the early stages of development when developers need to quickly test and iterate their applications. They allow developers to test how an app would run on different devices and operating systems without the need for actual hardware.

3. Cost-Effective:

Using emulators can be a cost-effective approach because it reduces the need for multiple physical devices during testing. Developers can emulate different devices and configurations on a single computer, saving on hardware costs.

4. **Limitations:**

While emulators provide a valuable testing environment, they do not perfectly replicate the hardware performance of actual devices. Issues such as differences in processing power, battery usage, and other hardware-specific characteristics might not be accurately reflected. Therefore, while emulators are good for initial testing, it's important to also test applications on physical devices to ensure they perform well in real-world conditions.

5. **Popular Emulators:**

There are several popular emulators used in mobile development. For Android development, Google provides an official Android Emulator that is part of the Android Studio IDE. Apple offers the Simulator app as part of Xcode for iOS development. These tools support a variety of device types and configurations.

b.SDK

SDK stands for Software Development Kit. An SDK is a toolkit that helps developers create applications, add new functionality, or make integrations. These specialty packages vary by form and function in each case but are typically downloadable and conform to a particular language or platform, offering reusable code components to make engineering tasks simpler.

SDKs may be platform-specific, such as supporting native mobile experiences (Android SDK, iOS SDK, Windows SDKs) or web-based frameworks (JavaScript, React Native, Flutter). Many SDKs are also language-specific, offering support for programming languages such as C#, Java, Node.js, PHP, Python, Ruby, Go, and others. These are often referred to as server-side SDKs or code libraries. API-first Software-as-a-Service companies regularly deploy SDKs to improve the integration developer experience.

So, what is *in* an SDK? An SDK may be comprised of code, knowledge, and interactive capabilities:

- Helper code libraries and reusable methods
- Reference documentation
- Set-up and authentication instructions
- Code samples, sample projects, or project templates
- Virtual testing environments or sandboxes
- Command-Line Interface (CLI) utilities
- UI design components
- Additional support materials

c.intents

Android Intent is the message that is passed between components such as activities, content providers, broadcast receivers, services etc. It is generally used with `startActivity()` method to invoke activity, broadcast receivers etc. The dictionary meaning of intent is intention or purpose. So, it can be described as the intention to do action.

Android intents are mainly used to:

- Start the service
- Launch an activity
- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.

Types of Intents

There are two primary types of Intents in Android:

1. Explicit Intents

Explicit intents are used when you want to start a specific component (such as an activity or service) directly within your application or another application where you know the exact class name of the component to be launched. This type of intent is very direct; you specify which component to start by providing its fully qualified class name.

2. Implicit Intents

Implicit intents, unlike explicit ones, do not specify the component. Instead, they declare a general action to perform, which allows any app component that filters for that action (and satisfies other criteria like data type) to handle the intent. This is useful for integrating with components of other applications without direct interaction with their internal structure.

d.Toast notification

A Toast notification is a small message that briefly appears on the screen of an Android device. It's used in apps to display quick alerts that inform the user but don't require them to take any action—meaning the app doesn't pause or wait for the user to tap on the notification. Instead, it shows up over whatever they're doing, and after a few seconds, it disappears on its own.

How Does a Toast Work?

Duration:

A Toast can last either a short time (about 2 seconds) or a longer time (about 3.5 seconds). You decide how long it should stay visible when you create it.

Position:

While Toasts typically pop up at the bottom of the screen, you can change this. If you prefer, you can make them appear at the top or in the middle of the screen.

No Interaction:

The user doesn't need to interact with a Toast. There's no need to dismiss it; it goes away by itself. This makes Toasts unobtrusive; they tell the user something useful but don't get in the way of their experience.

Example of Using a Toast:

```
Toast.makeText(context, "Your file has been saved", Toast.LENGTH_SHORT).show();
```

E.JSON

JSON, which stands for JavaScript Object Notation, is a lightweight data-interchange format that is easy for humans to read and write, and easy for machines to parse and generate. JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others.

Here's a breakdown of its main features:

1. **Text-based and Language-Independent:** JSON is a plain text format, which makes it easy to send across networks and to work with in any programming language.
2. **Structured Data:** JSON is built on two structures:
 - **Objects:** An object is an unordered set of name/value pairs. Objects are delimited by curly braces (`{}`) and contain keys and values separated by a colon (`:`). Each key-value pair is separated by a comma. This is similar to dictionaries in Python, objects in JavaScript, or maps in other languages.
 - **Arrays:** An array is an ordered list of values. Arrays are enclosed in square brackets (`[]`), and the values within them are separated by commas. This structure is similar to lists in Python or arrays in other programming languages.
3. **Data Types:** JSON supports various data types including strings, numbers, booleans (true/false), arrays, objects, and **null**. Strings in JSON must be enclosed in double quotes.
4. **Usage:** JSON is widely used for APIs and config files due to its simplicity and readability. It is commonly used in web applications for data exchange between clients and servers.

Example

```
{
  "name": "John Doe",
  "age": 30,
  "isStudent": false,
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212-555-1234"
    },
    {
      "type": "office",
      "number": "646-555-4567"
    }
  ],
  "address": {
    "street": "1234 Park Ave",
    "city": "New York",
    "state": "NY"
  }
}
```

Describe Float layout, Grid layout and Refresh layout in KivyMD. How are they used?

KivyMD, an extension of the Kivy framework, offers various layout options to manage the positioning and sizing of user interface elements in mobile and desktop applications. Each layout has its own unique features and best use cases. Here, we'll explore three specific types of layouts provided by KivyMD: FloatLayout, GridLayout, and RefreshLayout.

1. FrameLayout (Similar to FloatLayout in KivyMD)

FrameLayout is used to stack child views one above the other. This layout is the simplest one and is useful when you need to overlay several views in a single screen space.

```
// MainActivity.java
package com.example.myapp;

import android.os.Bundle;
import android.widget.Button;
import android.widget.FrameLayout;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        FrameLayout frameLayout = new FrameLayout(this);
        Button button = new Button(this);
        button.setText("Click Me");
        FrameLayout.LayoutParams params = new FrameLayout.LayoutParams(
            FrameLayout.LayoutParams.WRAP_CONTENT,
            FrameLayout.LayoutParams.WRAP_CONTENT
        );
        params.gravity = Gravity.CENTER;
        button.setLayoutParams(params);
        frameLayout.addView(button);

        setContentView(frameLayout);
    }
}
```

2. GridLayout

GridLayout is a layout that places its children in a rectangular grid. It is a very efficient way to arrange buttons in a calculator or elements in a dashboard.

```
package com.example.myapp;

import android.os.Bundle;
import android.widget.Button;
import android.widget.GridLayout;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    GridLayout gridLayout = new GridLayout(this);
    gridLayout.setColumnCount(2);
    gridLayout.setRowCount(2);

    for (int i = 0; i < 4; i++) {
        Button button = new Button(this);
        button.setText("Button " + (i + 1));
        gridLayout.addView(button);
    }

    setContentView(gridLayout);
}
}

```

3. SwipeRefreshLayout (Similar to RefreshLayout in KivyMD)

SwipeRefreshLayout is used to implement the pull-to-refresh pattern, commonly used with lists or any scrollable views.

```

// MainActivity.java
package com.example.myapp;

import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import androidx.appcompat.app.AppCompatActivity;
import androidx.swiperefreshlayout.widget.SwipeRefreshLayout;

public class MainActivity extends AppCompatActivity {

    SwipeRefreshLayout swipeRefreshLayout;
    ListView listView;
    String[] items = {"Item 1", "Item 2", "Item 3", "Item 4"};

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        swipeRefreshLayout = findViewById(R.id.swipeRefresh);
        listView = findViewById(R.id.listView);
        ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
        android.R.layout.simple_list_item_1, items);
        listView.setAdapter(adapter);
    }
}

```

```

swipeRefreshLayout.setOnRefreshListener(new SwipeRefreshLayout.OnRefreshListener() {
    @Override
    public void onRefresh() {
        // Refresh items
        refreshItems();
    }
});

void refreshItems() {
    // Load items
    // Items load complete
    onItemsLoadComplete();
}

void onItemsLoadComplete() {
    // Update the adapter and notify data set changed
    // Stop refresh animation
    swipeRefreshLayout.setRefreshing(false);
}
}

```

XML Layout (activity_main.xml):

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.swiperefreshlayout.widget.SwipeRefreshLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/swipeRefresh"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

</androidx.swiperefreshlayout.widget.SwipeRefreshLayout>

```

Explain the procedure to publish an App.

To publish an Android application, you need to follow a series of steps that involve preparing the app for release, testing it thoroughly, and finally releasing it to users.

Preparing Your App for Release:

1. Configure Your App: Disable logging, set debuggable to false, and update version information.

2. Build and Sign a Release Version: Use Gradle build files to create a signed APK.
3. Test Your App: Ensure proper functionality on different devices using Firebase Test Lab.
4. Update App Resources: Include multimedia files and graphics in your app.
5. Prepare Remote Servers: Ensure external servers are secure and ready for production.

Releasing Your App:

1. Choose Distribution Method:

Release through Google Play or other platforms.

2. Release on Google Play:

- Prepare promotional materials like screenshots and videos.
- Upload the release version of your app and publish it.

Mobile Application Development (Important Questions)

Q#1 What is Android? List any five Android SDK features and also Explain Android Architecture with neat diagram?

Q#2 What is Android Components and also explain with example?

Q#3 What is activity and also explain its five states? Explain life cycle of an activity.

Q#4 What is fragmentation and also explain the types of fragmentation and explain android fragment life cycle and its diagram and also describe the fragment transition and its type?

Q#5 Define Intent? Explain different types of intents along with their usage, demonstrate with code snippet how to pass data between activities with Intent using Intent Extra?

Q# 6 List and Explain different layout view and view group?

Q#7 Define Services? Explain types of Services? With a neat diagram explain the life cycle of Services?

Q#8 Explain the purpose of BroadCast Intent? Explain the different types of BroadCast Intent?

Q#9 List the different input controls for making choices? Demonstrate with a code snippet the usage of RadioButton/ Checkbox/ Spinner/ Alert Dialog/ Date Picker

Q#10 Define RecyclerView? Demonstrate with a code snippet the usage of RecyclerView?

Q#11 Define Shared Preferences? Differentiate between Shared Preferences and Saved Instance State?

Q#12 Define Content Provider? Explain with an example sharing the data between application using Content Provider?

Q#13 Differentiate between following: -

- Differentiate between Dalvik Virtual Machine vs Android Runtime
- Differentiate between activity and fragment?
- Differentiate between started service and bound service?

Q#14 Write short note:

- Manifest file
- Thread and Try-Catch
- Event listener interfaces
- SQLite database / Firebase / JSON
- Define APK?

What is Android? List any five Android SDK features and also Explain Android Architecture with neat diagram?

What is android? Discuss features of android.

Android is an open source and Linux-based Operating System for mobile devices such as smartphones and tablet computers. Android was developed by the Open Handset Alliance, led by Google, and other companies. Android offers a unified approach to application development for mobile devices which means developers need only develop for Android, and their applications should be able to run on different devices powered by Android.

The first beta version of the Android Software Development Kit (SDK) was released by Google in 2007 where as the first commercial version, Android 1.0, was released in September 2008.

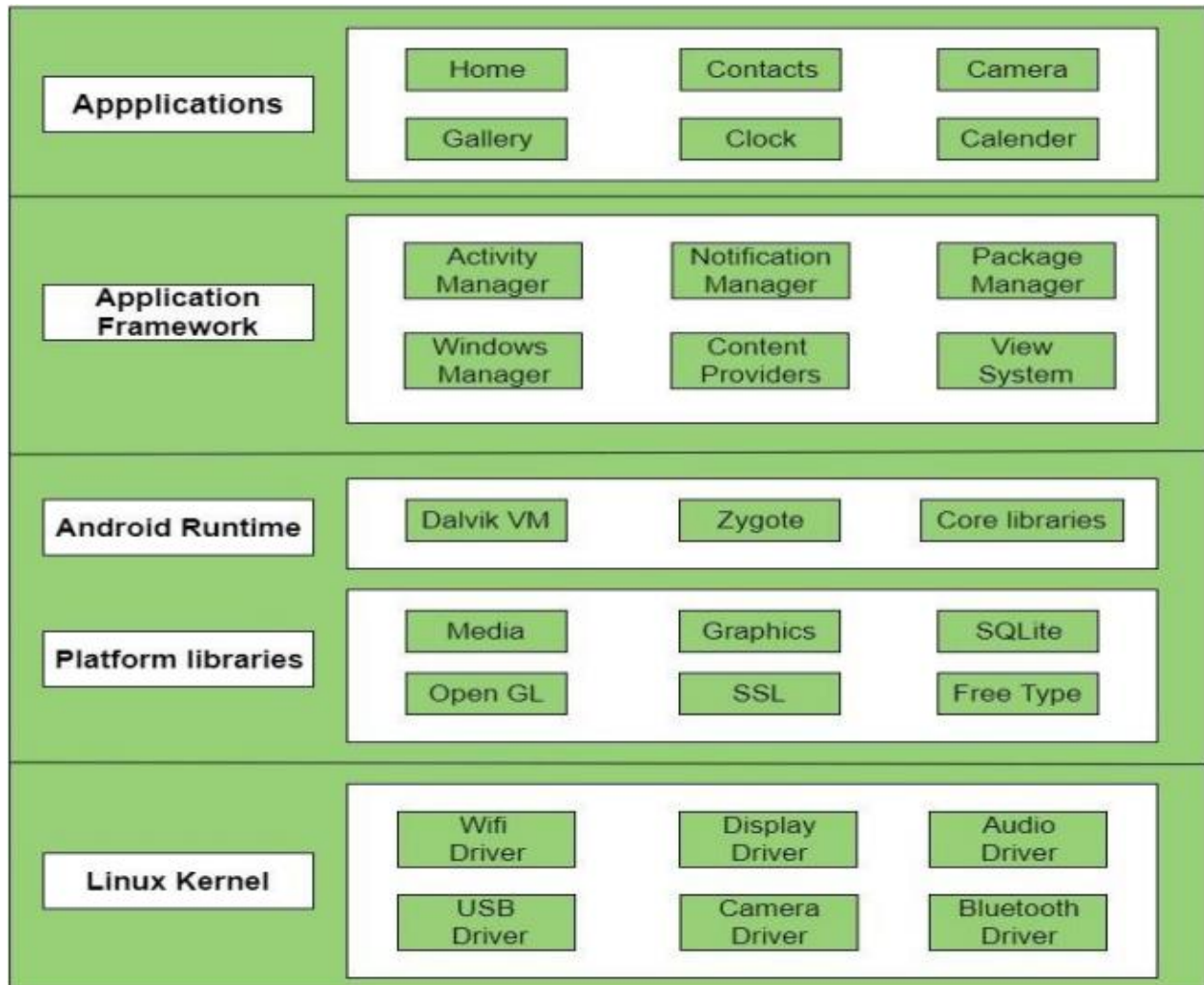
five Android SDK

The Android SDK (Software Development Kit) offers a range of features that enable developers to create applications for Android devices. Here are five notable features:

1. **Emulator:** The Android Emulator simulates Android devices on your computer, allowing developers to design, develop, and test applications without needing a physical device. It can emulate different configurations and features, including different screen sizes, Android versions, and hardware sensors.
2. **Android Studio Integration:** Android Studio is the official integrated development environment (IDE) for Android development. It provides tools for coding, debugging, and testing applications, including a rich editor, a set of developer tools, and a flexible build system.
3. **UI Design Tools:** The SDK includes tools like Layout Editor and ConstraintLayout, which help developers build complex user interfaces without needing to write extensive XML layout code. These tools offer a visual design interface that simplifies the process of UI creation.
4. **API Libraries:** Android SDK provides a vast set of APIs that allow developers to interact with the underlying Android system, including components for building user interfaces, accessing the file system, managing network connections, and integrating with Google services like Maps and Firebase.
5. **Profiling and Debugging Tools:** Tools such as Android Profiler and Logcat allow developers to monitor resource usage (like CPU, memory, and network usage) in real-time and log system output, respectively. These tools are crucial for optimizing app performance and troubleshooting issues during development.

Android Architecture

Android architecture is a software stack of components to support mobile device needs. Android software stack contains a Linux Kernel, collection of c/c++ libraries which are exposed through an application framework services, runtime, and application. Following are main components of android architecture those are.



Linux kernel :

This is the kernel on which Android is based. This layer contains all the low level device drivers for the various hardware components of an Android device.

Platform Libraries

These contain all the code that provides the main features of an Android OS. For example, the SQLite library provides database support so that an application can use it for data storage. The WebKit library provides functionalities for web browsing.

Android runtime

At the same layer as the libraries, the Android runtime provides a set of core libraries that enable developers to write Android apps using the Java programming language. The Android runtime also includes the Dalvik virtual machine, which enables every Android application to run in its own process. Dalvik is a specialized virtual machine designed specifically for Android and optimized for battery-powered mobile devices with limited memory and CPU.

Application framework

Exposes the various capabilities of the Android OS to application developers so that they can make use of them in their applications.

Applications

At this top layer, you will find applications that send with the Android device (such as Phone, Contacts, Browser, etc.), as well as applications that you download and install from the Android Market. Any applications that you write are located at this layer.

Q.#2 What is Android Components and also explain with example?

Application components are the essential building blocks of an Android application. These components are loosely coupled by the application manifest file `AndroidManifest.xml` that describes each component of the application and how they interact.

There are following four main components that can be used within an Android application

Activities

An activity represents a single screen with a user interface, in short Activity performs actions on the screen. For example, an email application might have one activity that shows a list of new emails, another activity to compose an email, and another activity for reading emails. If an application has more than one activity, then one of them should be marked as the activity that is presented when the application is launched.

An activity is implemented as a subclass of **Activity** class as follows –

```
public class MainActivity extends Activity {  
  
}
```

Services

A service is a component that runs in the background to perform long-running operations. For example, a service might play music in the background while the user is in a different application, or it might fetch data over the network without blocking user interaction with an activity.

A service is implemented as a subclass of **Service** class as follows –

```
public class MyService extends Service {  
}
```

Broadcast Receivers

Broadcast Receivers simply respond to broadcast messages from other applications or from the system. For example, applications can also initiate broadcasts to let other applications know that some data has been downloaded to the device and is available for them to use, so this is broadcast receiver who will intercept this communication and will initiate appropriate action.

A broadcast receiver is implemented as a subclass of **BroadcastReceiver** class and each message is broadcaster as an **Intent** object.

```
public class MyReceiver extends BroadcastReceiver {  
    public void onReceive(context,intent){}  
}
```

Content Providers

A content provider component supplies data from one application to others on request. Such requests are handled by the methods of the *ContentResolver* class. The data may be stored in the file system, the database or somewhere else entirely.

A content provider is implemented as a subclass of **ContentProvider** class and must implement a standard set of APIs that enable other applications to perform transactions.

```
public class MyContentProvider extends ContentProvider {  
    public void onCreate(){  
    }  
}
```

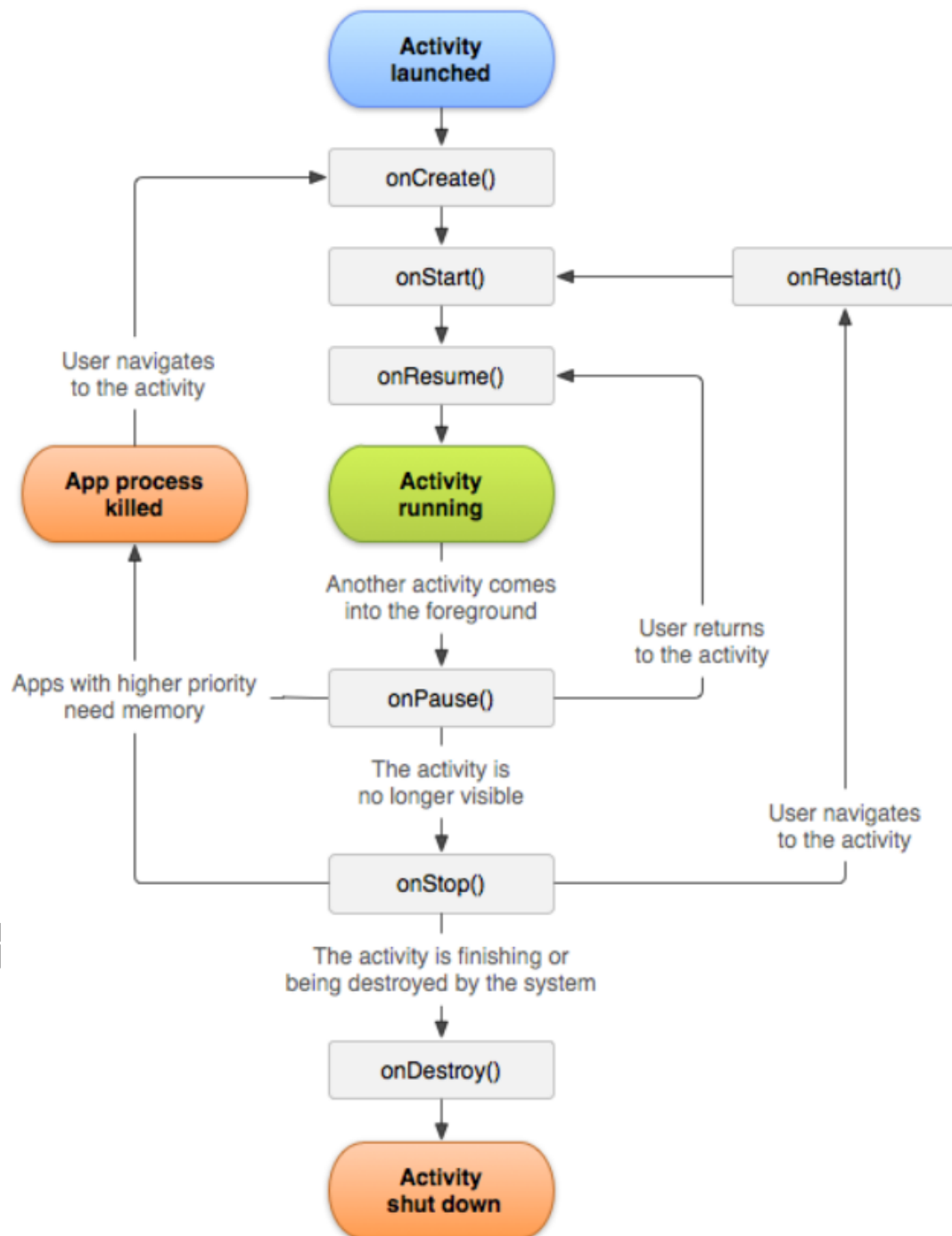
Q.#3 What is activity and also explain its five states? Explain life cycle of an activity.

An activity is one screen of an app. In that way the activity is very similar to a window in the Windows operating system. The most specific block of the user interface is the activity. An Android app contains activities, meaning one or more screens.

Examples: Login screen, sign up screen, and home screen.

LIFECYCLE OF ACTIVITY

An activity goes through a number of states. Activity lifecycle in Android manages the different states of activity, such as when the activity starts or stops, etc. All of these states are managed by callback methods.



There are seven callback methods:-

onCreate():- This callback method must be created by you, this is fired when your activity is created by the system. For example, when you open the app it will call onCreate(), onStart(), and onResume() methods. These three methods are initiated when you open an app.

onStart():- There should always be an onStart() callback after onCreate() finishes. In the started state users will be able to see the activity but they are not ready for user interaction.

onResume():- In this, activities will be in the foreground and ready for user interaction. You can understand the use of this callback from this example: "when the user presses the phone's answer button it will give onPause() after you end up calling it will again give onResume()".

onPause():- This callback occurs when there is a case of another activity starting in front of the current activity. It is the opposite of onResume(). An example for this callback can be "when you open the app, that is another app from the notification bar, or open settings then it will call onPause() and onStop()".

onStop():- It is the opposite of onStart(). In this case, activity will no longer be visible to the users.

onRestart():- This occurs when the activity is stopped and before the activity starts again. It is a less common callback.

onDestroy():- It is opposite of onCreate(). This starts when the system needs to free memory or when finish() is called. It is used for cleanup which is a very common activity.

```
import android.os.Bundle;
import android.app.Activity;
import android.util.Log;

public class MainActivity extends Activity {
    String msg = "Android : ";

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(msg, "The onCreate() event");
    }

    @Override
    protected void onStart() {
        super.onStart();
        Log.d(msg, "The onStart() event");
    }
}
```

```

    }

    @Override
    protected void onResume() {
        super.onResume();
        Log.d(msg, "The onResume() event");
    }

    @Override
    protected void onPause() {
        super.onPause();
        Log.d(msg, "The onPause() event");
    }

    @Override
    protected void onStop() {
        super.onStop();
        Log.d(msg, "The onStop() event");
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        Log.d(msg, "The onDestroy() event");
    }
}

```

Q.#4 What is fragmentation and also explain the types of fragmentation and explain android fragment life cycle and its diagram and also describe the fragment transition and its type?

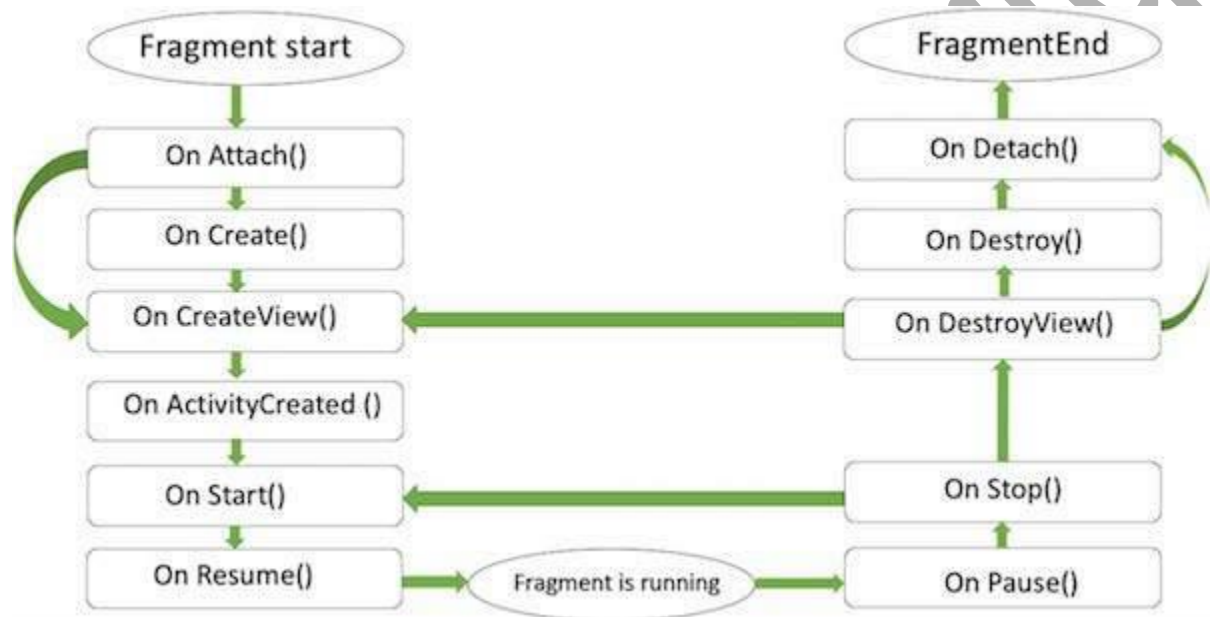
A Fragment is a piece of an activity which enable more modular activity design. It will not be wrong if we say, a fragment is a kind of sub-activity.

Following are important points about fragment

- A fragment has its own layout and its own behaviour with its own life cycle callbacks.
- You can add or remove fragments in an activity while the activity is running.

- You can combine multiple fragments in a single activity to build a multi-pane UI.
- A fragment can be used in multiple activities.
- Fragment life cycle is closely related to the life cycle of its host activity which means when the activity is paused, all the fragments available in the activity will also be stopped.
- A fragment can implement a behaviour that has no user interface component.
- Fragments were added to the Android API in Honeycomb version of Android which API version 11.

Fragment Life Cycle



Fragment lifecycle

Here is the list of methods which you can to override in your fragment class –

- **onAttach()** The fragment instance is associated with an activity instance. The fragment and the activity is not fully initialized. Typically you get in this method a reference to the activity which uses the fragment for further initialization work.
- **onCreate()** The system calls this method when creating the fragment. You should initialize essential components of the fragment that you want to retain when the fragment is paused or stopped, then resumed.
- **onCreateView()** The system calls this callback when it's time for the fragment to draw its user interface for the first time. To draw a UI for your fragment, you must return a **View** component from this method that is the root of your fragment's layout. You can return null if the fragment does not provide a UI.

- **onActivityCreated()** The onActivityCreated() is called after the onCreateView() method when the host activity is created. Activity and fragment instance have been created as well as the view hierarchy of the activity. At this point, view can be accessed with the findViewById() method. example. In this method you can instantiate objects which require a Context object
- **onStart()** The onStart() method is called once the fragment gets visible.
- **onResume()** Fragment becomes active.
- **onPause()** The system calls this method as the first indication that the user is leaving the fragment. This is usually where you should commit any changes that should be persisted beyond the current user session.
- **onStop()** Fragment going to be stopped by calling onStop()
- **onDestroyView()** Fragment view will destroy after call this method
- **onDestroy()** onDestroy() called to do final clean up of the fragment's state but Not guaranteed to be called by the Android platform.

Types of Fragments

Basically fragments are divided as three stages as shown below.

Single frame fragments

Single frame fragments are using for hand hold devices like mobiles, here we can show only one fragment as a view. Single frame fragment is designed for small screen devices such as hand hold devices(mobiles) and it should be above android 3.0 version.

List fragments

fragments having special list view is called as list fragment. The basic implementation of list fragment is for creating list of items in fragments



Fragments transaction

Activity and Fragment transitions in Lollipop are built on top of a relatively new feature in Android called Transitions. Introduced in KitKat, the transition framework provides a convenient API for animating between different UI states in an application. The framework is built around two key concepts: scenes and transitions. A scene defines a given state of an application's UI, whereas a transition defines the animated change between two scenes.

When a scene changes, a Transition has two main responsibilities –

- Capture the state of each view in both the start and end scenes.
- Create an Animator based on the differences that will animate the views from one scene to the other.

Q.#5 Define Intent? Explain different types of intents along with their usage, demonstrate with code snippet how to pass data between activities with Intent using Intent Extra

Android Intent is the message that is passed between components such as activities, content providers, broadcast receivers, services etc. It is generally used with `startActivity()` method to invoke activity, broadcast receivers etc. The dictionary meaning of intent is intention or purpose. So, it can be described as the intention to do action.

Android intents are mainly used to:

- Start the service
- Launch an activity
- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.

Types of Intents

There are two primary types of Intents in Android:

1. Explicit Intents

Explicit intents are used when you want to start a specific component (such as an activity or service) directly within your application or another application where you know the exact class name of the component to be launched. This type of intent is very direct; you specify which component to start by providing its fully qualified class name.

2. Implicit Intents

Implicit intents, unlike explicit ones, do not specify the component. Instead, they declare a general action to perform, which allows any app component that filters for that action (and satisfies other criteria like data type) to handle the intent. This is useful for integrating with components of other applications without direct interaction with their internal structure.

Passing Data Between Activities Using Intent Extra

To demonstrate how to pass data between activities using **Intent** and **Intent Extra**, let's look at an example where we have two activities: **MainActivity** and **SecondActivity**. We will pass a string from **MainActivity** to **SecondActivity**.

Code Snippet

MainActivity.java - This is the source activity where the intent is created.

```
package com.example.myapp;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button button = (Button) findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                // Create an Intent to start SecondActivity
                Intent intent = new Intent(MainActivity.this, SecondActivity.class);

                // Add extra data to the Intent
                intent.putExtra("EXTRA_MESSAGE", "Hello from MainActivity!");

                // Start SecondActivity
                startActivity(intent);
            }
        });
    }
}
```

SecondActivity.java - This is the destination activity that receives the data.

```
package com.example.myapp;

import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
    }
}
```

```
// Get the Intent that started this activity
Intent intent = getIntent();
String message = intent.getStringExtra("EXTRA_MESSAGE");

// Capture the layout's TextView and set the string as its text
TextView textView = (TextView) findViewById(R.id.textview);
textView.setText(message);
}
}
```

Q.# 6 List and Explain different layout view and view group?

In Android development, the layout view and view groups are fundamental components for creating user interfaces. Here's a detailed explanation of the different layout views and view groups commonly used:

1. Layout Views

These are basic building blocks for user interface components in Android. Layout views define the visual structure for a user interface, such as the arrangement of buttons, text fields, and other widgets. Here are some of the primary layout views:

- **LinearLayout:** This layout arranges its children in a single direction, either vertically or horizontally. You can control the distribution of space among children using weights.
- **RelativeLayout:** This allows positioning the children relative to each other (e.g., to the right of or below another child) or to the parent (e.g., aligned to the right edge of the layout).
- **FrameLayout:** Designed to block out an area on the screen to display a single item. You can stack multiple children, and each child can be positioned based on the "gravity" attribute.
- **GridLayout:** This layout views the space into a grid of cells. Each component in a GridLayout is placed in a rectangular area of the grid, and the grid is defined by its rows and columns.
- **ConstraintLayout:** A more flexible and complex layout that allows you to create large and complex layouts with a flat view hierarchy (no nested view groups). It provides various constraints for positioning and sizing widgets in relation to other elements in the layout.

2. View Groups

View groups are containers that hold views (i.e., widgets) and define their layout properties. Each of the layout views described above is actually a type of view group, as they can contain other views. However, there are additional view groups that are used for specific purposes:

- **ScrollView:** A view group that allows the view it contains to be scrolled vertically.

- **HorizontalScrollView:** Similar to ScrollView, but it allows horizontal scrolling instead.
- **NavigationView:** Often used in Android Navigation Drawer pattern, it provides a sliding menu that appears from the side of the main content area, containing a navigation menu.
- **ViewPager:** A view group that allows the user to flip left and right through pages of data. Often used in conjunction with fragments, this is useful for implementing multi-page user interfaces.
- **DrawerLayout:** This acts as a root-level container that adds a drawer for navigation options on the left or right side of the window.
- **CoordinatorLayout:** An enhanced FrameLayout that works with specific child views to coordinate their animations or layout behavior.

Q.#7 Define Services? Explain types of Services? With a neat diagram explain the life cycle of Services?

A service is a component that runs in the background to perform long-running operations without needing to interact with the user and it works even if application is destroyed. A service can essentially take two states

Types of Services

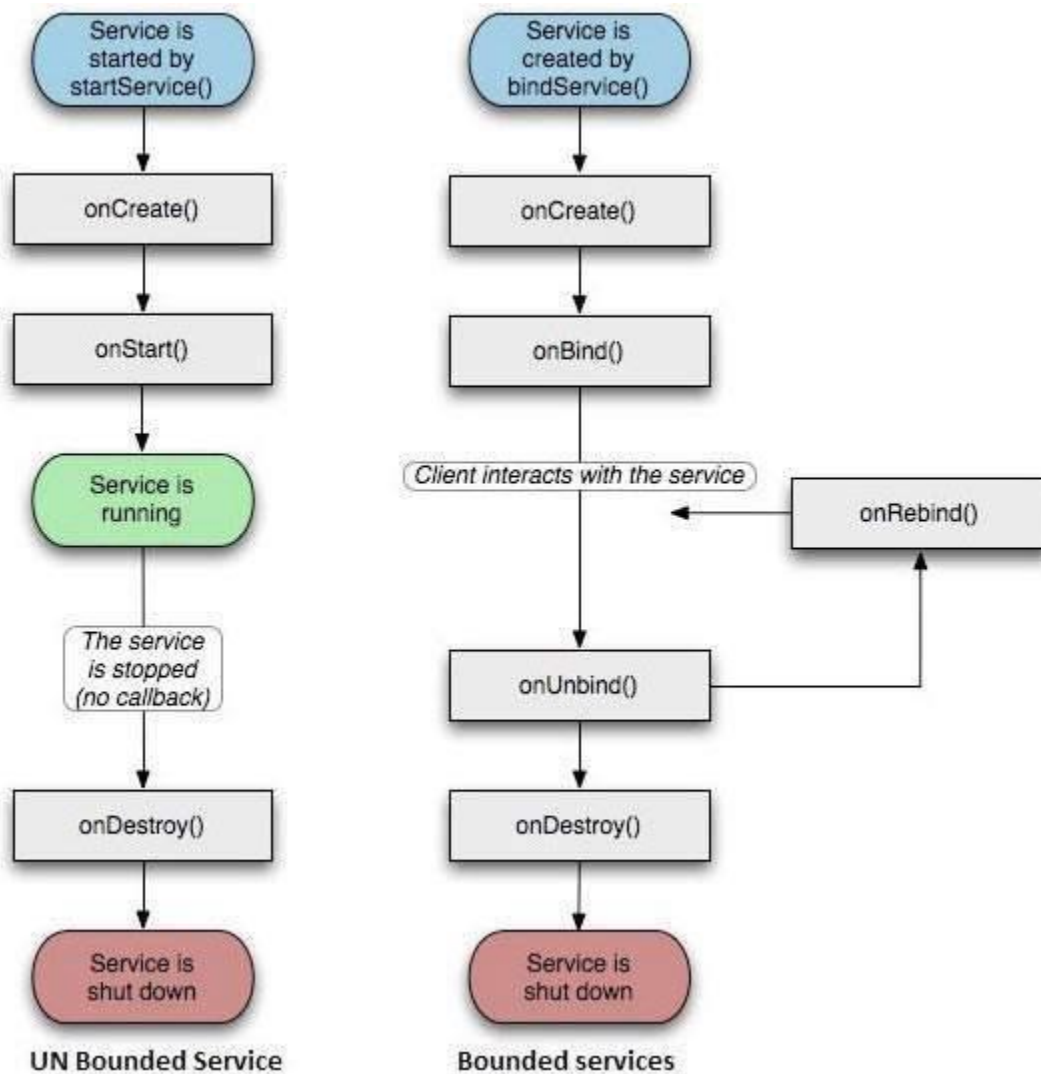
Started

A service is started when an application component, such as an activity, starts it by calling `startService()`. Once started, a service can run in the background indefinitely, even if the component that started it is destroyed.

Bound

A service is bound when an application component binds to it by calling `bindService()`. A bound service offers a client-server interface that allows components to interact with the service, send requests, get results, and even do so across processes with interprocess communication (IPC).

Life Cycle



`onStartCommand()`

The system calls this method when another component, such as an activity, requests that the service be started, by calling `startService()`. If you implement this method, it is your responsibility to stop the service when its work is done, by calling `stopSelf()` or `stopService()` methods.

`onBind()`

The system calls this method when another component wants to bind with the service by calling `bindService()`. If you implement this method, you must provide an interface that clients use to communicate with the service, by returning an `IBinder` object. You must always implement this method, but if you don't want to allow binding, then you should return null.

`onUnbind()`

The system calls this method when all clients have disconnected from a particular interface published by the service.

onRebind()

The system calls this method when new clients have connected to the service, after it had previously been notified that all had disconnected in its onUnbind(Intent).

onCreate()

The system calls this method when the service is first created using onStartCommand() or onBind(). This call is required to perform one-time set-up.

onDestroy()

The system calls this method when the service is no longer used and is being destroyed. Your service should implement this to clean up any resources such as threads, registered listeners, receivers, etc.

Q.#8 Explain the purpose of Broadcast Intent? Explain the different types of Broadcast Intent?

In Android development, a Broadcast Intent is a mechanism used to communicate information across different parts of an Android application or even across different applications. This allows an app to react to system-wide or application-specific events. For example, apps can listen for broadcasts that notify them of changes like network changes, battery level changes, or when the user has booted the device.

Purpose of Broadcast Intent

1. **Event Distribution:** Broadcast Intents are used to distribute an event to all interested receivers. These events can originate from the system (like battery low, screen turned off) or from applications (such as notifications or data updates).
2. **Decoupled Communication:** Broadcasts enable communication between components without requiring the components to explicitly know about each other. This decoupling of components and apps enhances modularity and flexibility in application design.
3. **System Integration:** They provide a way for apps to integrate closely with the Android system. Apps can listen for specific system events (like installation of a new app, timezone change) to perform appropriate actions.
4. **Trigger Actions:** Apps can trigger actions that other apps can listen to and respond accordingly, facilitating a reactive programming style.

Types of Broadcast Intents

There are generally two types of Broadcast Intents in Android:

Standard Broadcasts (Normal Broadcasts):

These are asynchronous broadcasts that the Android system delivers to all receivers in an undefined order. This type is suitable for lower-priority notifications that don't require the receiver to return a result or process the broadcast immediately.

Example: `Intent.ACTION_TIME_TICK` which notifies every minute.

Ordered Broadcasts:

These are broadcasts that are sent to one receiver at a time. Android can deliver these broadcasts to receivers in an order defined by the priority specified in their intent filters. One receiver can propagate a result to the next, or completely abort the broadcast so it won't be passed to other receivers.

Example: SMS messages are handled this way, where an SMS app can abort the broadcast to prevent other apps from reading the SMS if needed.

Sticky Broadcasts: (deprecated in API level 21 due to privacy concerns)

Sticky broadcasts are those broadcasts that remain in the system after being broadcast. The state of the broadcast can be queried or accessed by apps after it occurs and not just during the broadcast delivery.

Example: `Intent.ACTION_BATTERY_CHANGED` is a sticky broadcast that contains the battery charge level and its charging state.

Local Broadcasts:

Managed within the app itself without affecting other applications. This is achieved using the `LocalBroadcastManager` which allows you to register for and send broadcasts of intents to local receivers within your app safely and efficiently.

Example: Communicating between different components of the same application without the need for the system's intervention.

Q.#9 List the different input controls for making choices? Demonstrate with a code snippet the usage of RadioButton/ Checkbox/ Spinner/ Alert Dialog/ Date Picker

In Android development, there are several input controls available for making choices. These controls enable developers to design interactive user interfaces that can capture user preferences and inputs. Here, I'll discuss some of the common controls such as `RadioButton`, `Checkbox`, `Spinner`, `Alert Dialog`, and `Date Picker`, along with code snippets demonstrating their usage in an Android application.

1. RadioButton

`RadioButtons` are used when you want users to select only one option from a set. They are usually grouped inside a **`RadioGroup`**.

XML Layout:

```
<RadioGroup
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <RadioButton
        android:id="@+id/radioButton1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Option 1" />
    <RadioButton
        android:id="@+id/radioButton2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Option 2" />
</RadioGroup>
```

2. Checkbox

Checkboxes allow users to select one or more options independently.

XML Layout:

```
<CheckBox
    android:id="@+id/checkbox1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Option A" />
```

3. Spinner

Spinners provide a dropdown menu for selecting one item from a set.

XML Layout:

```
<Spinner
    android:id="@+id/spinner"
    android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content" />
```

4. Alert Dialog

Alert Dialogs can offer choices or information to users.

```
AlertDialog.Builder builder = new AlertDialog.Builder(context);
```

```
builder.setTitle("Title")
```

```
.setMessage("Dialog message")
```

```
.setPositiveButton("OK", new DialogInterface.OnClickListener() {
```

```
    public void onClick(DialogInterface dialog, int id) {
```

```
        // User clicked OK button
```

```
    }
```

```
});
```

```
.setNegativeButton("Cancel", new DialogInterface.OnClickListener() {
```

```
    public void onClick(DialogInterface dialog, int id) {
```

```
        // User cancelled the dialog
```

```
    }
```

```
});
```

```
AlertDialog dialog = builder.create();
```

```
dialog.show();
```

5. Date Picker

Date Pickers allow users to select a date.

XML Layout:

No XML layout required for basic usage; it's typically triggered from a button click.

```
DatePickerDialog datePickerDialog = new DatePickerDialog(context,
```

```
new DatePickerDialog.OnDateSetListener() {
```

```
    @Override
```

```
    public void onDateSet(DatePicker view, int year, int month, int dayOfMonth) {
```

```
        // handle the date selected
```

```
    }
```

```
}, year, month, day);
```

```
datePickerDialog.show();
```

Q.#10 Define RecyclerView? Demonstrate with a code snippet the usage of RecyclerView?

RecyclerView is a flexible and efficient version of ListView used in Android development. It's part of the Android Support Library and is designed to display large sets of data in your app while using minimal system resources. RecyclerView recycles and reuses cells that scroll off-screen to save memory and ensure smooth, fast scrolling, regardless of the complexity or size of the list.

Key Components of RecyclerView

- **Adapter:** Manages the data model and binds data to views that are displayed within the RecyclerView.
- **LayoutManager:** Handles the layout of individual elements in the RecyclerView, including linear, grid, or staggered grid arrangements.
- **ItemDecoration:** Allows the application of decorations around the items (e.g., dividers between items).
- **ItemAnimator:** Manages animations for item additions, removals, and changes.

Example Usage of RecyclerView

Here's a simple example to demonstrate the usage of **RecyclerView** in an Android application. This example includes a **RecyclerView** that displays a list of strings.

First, define the RecyclerView in your XML layout file.

```
<!-- res/layout/activity_main.xml -->
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</RelativeLayout>
```

Q.#11 Define Shared Preferences? Differentiate between Shared Preferences and Saved Instance State?

Shared Preferences is a mechanism in Android for storing and retrieving small amounts of data in the form of key-value pairs. This data persists across user sessions, even if the application is killed. Shared Preferences are typically used to store simple configuration details or user preferences, and they provide a convenient way to read and write primitive data types (such as int, float, long, string, and boolean). Data stored in Shared Preferences is not secure and can be accessed by any application with the appropriate permissions.

Feature	Shared Preferences	Saved Instance State
Persistence	Persistent across sessions until deleted or app uninstalled.	Temporary, only during activity lifecycle.
Scope	Used for storing user settings and preferences.	Used to save UI state and transient activity data.
Accessibility	Accessible from any part of the app and other apps if not private.	Accessible only within the activity or fragment where it is stored.
Security	Data is stored in plain text unless explicitly encrypted.	Security is minimal as it's not meant for long-term storage.
Use Case	Storing preferences like notification settings, theme settings.	Maintaining UI state during configuration changes such as screen rotations.

#12 Define Content Provider? Explain with an example sharing the data between application using Content Provider?

Content Providers

A content provider component supplies data from one application to others on request. Such requests are handled by the methods of the *ContentResolver* class. The data may be stored in the file system, the database or somewhere else entirely.

A content provider is implemented as a subclass of **ContentProvider** class and must implement a standard set of APIs that enable other applications to perform transactions.

```
public class MyContentProvider extends ContentProvider {  
    public void onCreate(){}  
}
```

```
}
```

How Content Providers Work

Content providers in Android operate through a set of standard APIs that handle data requests from other applications. These APIs include methods like **query()**, **insert()**, **delete()**, **update()**, and **getType()** which are used to perform operations on the data. The data itself is typically accessed via a URI scheme, which provides a standard way for identifiers to be assigned to the data.

Example of Sharing Data Between Applications Using Content Providers

Let's consider an example where there are two applications: **App A** (a contact management app) and **App B** (a social networking app). Both applications need access to the same set of user contacts data.

1. Creation of Content Provider in App A:

- App A, which manages contacts, implements a content provider to encapsulate the contacts data stored in an SQLite database. The content provider defines a URI like **content://com.appa.contactsprovider/contacts** for accessing the contact data.

2. Accessing Data from App B:

- App B, which needs to display the contacts, does not need to know how the data is stored. Instead, it interacts with the content provider through the URI. App B can query the contacts like this:

```
Uri contactsUri = Uri.parse("content://com.appa.contactsprovider/contacts");
Cursor cursor = getContentResolver().query(contactsUri, null, null, null, null);
if (cursor != null) {
    while (cursor.moveToNext()) {
        // Process each contact
    }
    cursor.close();
}
```

Reference sites(take help from):

Greeksforgreeks

GitHub

ChatGPT -4

Gemini Pro

(IF FIND ANY MISTAKE CONTACT ME)

- Mr.noman.tariq@outlook.com
- **03700204207(WhatsApp only)**